

DIFA-Rent: Division Property Based Fault Attacks on DEFAULT and BAKSHEESH

Shibam Ghosh¹ , Anup Kumar Kundu² , and Dhiman Saha³ 

¹Inria, Paris, France

shibam.ghosh@inria.fr

² Indian Statistical Institute, Kolkata 700108, India

anupkundumath@gmail.com

³de.ci.phe.red Lab, Department of Computer Science & Engineering,
Indian Institute of Technology Bhilai, Chhattisgarh - 491002, India

dhiman@iitbhillai.ac.in

Abstract. Fault attacks have historically been one of the most popular gray-box attacks in Cryptographic literature. In such attacks, an attacker tries to inject perturbations while executing a cipher and exploit the faulty outputs to recover the key. While the efficiency of such an attack is measured by number of faults required and size of the reduced key-space, another pivotal temporal parameter in the point of fault injection which has not received considerable attention. In this work, we plug this gap for a special class of ciphers namely DEFAULT and BAKSHEESH which boast of an SBox with one or more linear structures (LS). We make new observations which lead to the improvement of Division Property based fault attacks (DIFA) introduced by Kundu *et al.* in ACNS 2023. We show that these linear structures are particularly responsible for giving the higher fault penetration in these ciphers. We improve the state-of-the-art for BAKSHEESH from 30 to 28 rounds and for DEFAULT from 75 to 72 using the single round random nibble fault model. While for BAKSHEESH we are able to uniquely recover the key, for DEFAULT, we are able to reduce the key-space to 2^{64} . This leads to the best fault attacks on BAKSHEESH and DEFAULT in terms of number of rounds penetrated for fault injection. Our work reiterates the fact that a property induced (in this case LS in SBox) for some particular Cryptographic purposes (like fault attack resistance) may manifest orthogonally for another (increasing fault penetration) and thus adds value to block cipher design space exploration and fault attack counter-measure development.

1 Introduction

Modern Cryptography has evolved beyond the barriers of black-box models where only know of the algorithms is presumed. Gray-box models which additionally assume access to cipher implementations and ability to monitor side-channel activity and/or transiently modify cipher execution to observe fault ciphertexts have become increasingly popular given their practical relevance. Fault attacks (FA) have evolved to be one of the most talked about gray-box

attacks since the introduction of fault analysis in public key cryptography (RSA-CRT implementations by Boneh et al. [8] [9]) and subsequently in symmetric cryptography (DFA on DES by Biham and Shamir [6]). If we closely follow the ways that FA has evolved, it shows us two directions. In one direction, we have seen various interpretations of the nature of the faults induced. This includes but is not limited to Statistical Fault Analysis (SFA) [17], Collision Fault Analysis (CFA) [7], Persistent Fault Analysis (PFA) [34], Statistically Ineffective Fault Analysis (SIFA) [16], and Statistically Effective Fault Analysis (SEFA) [31]. The other direction of fault analysis evolution constitutes is used for developing new types of fault attacks, viz., Integral Fault Analysis (IFA) [27], Slow Diffusion Fault Analysis (SDFA) [14,1,21], Internal Differential Fault Analysis (InDFA) [26], Meet-in-the-Middle and Impossible Differential Fault Analysis (ImpDFA) [13], Truncated Differential based Fault Analysis (ToFA) [22], Division Property Based Fault Analysis (DiFA) [23].

As regards, measuring the efficiency of fault attacks, the main contending parameter has been the number of faults coupled with the fault model. The random byte model has been one of the well-adopted ones especially of byte-oriented ciphers. However, for the general consensus is word-oriented fault model. For instance for a nibble based designs, a random nibble fault model may be preferred. Another fault model which has been assumed universally is the random bit model with some criticism from fault injection experts citing that these fault may be too precise to be realized in practice in large numbers and they require higher precision. It has been shown time and again that the fault count is very closely related to the fault model. For instance, in case of an integral/division property based attacks the fault count is always a multiple of size of the minimal integral set being used to devise the attack. On a different note, the point of fault injection another point of interest. This choice can make or break an attack. If the injection is more closer to the end then, the fault may not diffuse enough and key-recovery is limited implying that the attack might need to be repeated multiple times. On the flip-side if the point of fault injection is too distant from the end, then the fault may diffuse in the state in a unexploitable manner. So, the contention is to be able to choose the sweet-spot to get the best attack in terms of fault count. However, there is a trade-off. It might be possible to inject the fault in an earlier round and still be able to attack albeit at a *higher* fault count. One might argue that this is not optimal. However, the argument in favor of that analysis is that *increasing fault penetration akin to higher round attack in classical cryptanalysis*. Additionally, showing attacks on earlier rounds also contributes to counter-measure development as it establishes from which round the counter-measure should come in effect. The current work is an attempt to get the best fault attacks on two lightweight ciphers DEFAULT [3] and BAKSHEESH [4] in terms of fault penetration.

The choice of these ciphers is not random but due to their choice of SBoxes. In both these designs, the authors have argued in favour of Linear Structures (LS) in the SBoxes chosen with one of the motivations being fault attack resistance. However, as per our observations, this choice itself is allowing us to

extend division trails by two rounds for BAKSHEESH and three rounds for DEFAULT. The new trails devised lead to a extension of the number of rounds of fault injection thereby improving the state-of-the-art. Additionally, our research highlights that the element that was introduced by the designers as a safeguard against fault attacks led to the facilitation of the extension of fault injection.

1.1 Related Work

In [22], the authors have introduced a novel fault attack strategy against bit oriented block ciphers like GIFT, BAKSHEESH by using the impossible differential characteristics. It is shown that, some truncated differential pattern appears after some intermediate round. Any difference which is outside of this permissible truncated difference, leads to an impossible differential for the cipher. In this work, they have injected some random byte fault specifically at $\mathcal{R}_{35}$ for GIFT-128 and $\mathcal{R}_{32}$ for BAKSHEESH cipher. This reduces the whole key space of the cipher to a unique one by injecting only 3 and 12 faults for GIFT-128 and BAKSHEESH respectively.

In [15], the authors present a DFA on the initial version of DEFAULT. This is one of the first work on this cipher, which challenged the security claim given by the designers. It is a multi round fault attack where the faults are given in the random nibbles at the last and the second last round. The total key space of the cipher reduces to 2^{16} using 112 number of faults.

The work in [25] presents an information combining DFA on DEFAULT. The designers of DEFAULT [3] have claimed that, the key space cannot be reduced more than 2^{64} using differential fault attack. Here the authors have combined information across multiple rounds to recover the full master key of the cipher for the rotating key schedule also. The attack first reduces the key space to a class of equivalent keys and then represent the equivalent keys in a normalized form using linear equations. They have given the simulation code and shown that with less than 100 faulty computations, the total key space can be reduced to a unique one using their attack approach.

Another DFA on both the ciphers DEFAULT and BAKSHEESH is given by Jana *et al.* [20] at Asiacrypt 2024. They have shown that, the deterministic trail up to five rounds can be found if the injected fault position is known to the attacker. Here the authors have injected 36 faults at the fifth to last round, to recover the unique key of DEFAULT-LAYER. For BAKSHEESH, the required number of faults is 12 at the third to last round. They have also proposed a novel attack idea, called SDFA by combining the classical statistical and differential fault attack models. Using SDFA, the master key can be recovered using 128 bit set faults in the worst case.

1.2 Our Contributions

One of the primary motivations of this work was to study how the linear structures in an SBox facilitate in extending fault based differentials, thereby increasing the penetration of the fault injection point. To exploit specific properties

of LS structured SBoxes, we studied the direction provided in [24] to extend division property based distinguishers by composing linear layers at the output. Based on the observations of linear structure effect, we were able to apply and extend the DiFA attack introduced by Kundu *et al.* in ACNS 2023 by **two** rounds in term of fault penetration and develop a similar attack on DEFAULT improving the fault penetration by **three** more rounds. This leads to the best fault attacks on BAKSHEESH and DEFAULT in terms of number of rounds penetrated for fault injection. A summary of our findings, together with comparisons to existing attacks on BAKSHEESH and DEFAULT, are presented in table 1.

Table 1: Summary of fault attacks on BAKSHEESH and DEFAULT

Primitive	Fault Model	#faults	FI Point	Key-Space	Ref	Remarks
BAKSHEESH	Random Byte	4	30	1	[22]	Single-Round Fault
	Random but Known Bit	12	32	1	[20]	
	Random Nibble	96	28	2^1	Sec. 4.1	
DEFAULT	Known single bit-flip	(84 ± 15)	74	1	[25]	Multi-Round Fault [†]
	Random Nibble	112	78, 79	2^{16}	[15]*	Multi-Round Fault
	Random bit-flip	36	75	1	[20]	Single-Round Fault
	Random Nibble	96	72	2^{64}	Sec. 4.2	

* Applied on the initial version of DEFAULT.

† The fault is given at 6 different rounds of the cipher.

Organization of the paper. Section 2 contains the notations which will be used throughout the paper. The specification of the ciphers and the background knowledge related to this work, is given in this section. In section 3, we have shown the behaviour of the BIZ property through LS structured SBox. The key recovery attack for 7 round of BAKSHEESH is given in section 4.1 and 8 round of DEFAULT is given in section 4.2. After giving the simulation part of the attack in section 5, we have concluded this work in section 6.

2 Preliminaries and Background

2.1 Notations

In this section, we have introduced the notations that will be used later in this work. The vectors in a binary field are denoted by using bold lowercase letters (e.g. $\mathbf{x}$). The i -th coordinate of an n -bit vector $\mathbf{x} \in \mathbb{F}_2^n$, is represented by x_i

and written as, $\mathbf{x} = (x_{n-1}, \dots, x_0)$. The binary vector with all elements 0, is represented by $\mathbf{0}$. Let $Y \subseteq \mathbb{F}_2^n$, is a multi-set of vectors. A coordinate position i , with $0 \leq i < n$, is said to be a balanced bit position of Y , if $\bigoplus_{\mathbf{y} \in Y} y_i = 0$. Given two vectors $\mathbf{u}, \mathbf{x} \in \mathbb{F}_2^n$, we define the monomial $\mathbf{x}^{\mathbf{u}}$ as $\mathbf{x}^{\mathbf{u}} = \prod_{i=0}^{n-1} x_i^{u_i}$, and $\mathbf{x} \cdot \mathbf{y} = \bigoplus_{i=0}^{n-1} x_i \cdot y_i$. Given two vectors $\mathbf{k}, \mathbf{x} \in \mathbb{F}_2^n$, we define $\mathbf{x} \succeq \mathbf{k}$ if $x_i \geq k_i$ for all $0 \leq i < n$. Circular left shift of $\mathbf{x} \in \mathbb{F}_2^{128}$ by n bit, is denoted by $\mathbf{x} \lll n$.

2.2 Bit Permutation Based SPN Block Cipher

In this paper, we study DEFAULT [3] and BAKSHEESH [4] ciphers. Both of these ciphers are substitution permutation network (SPN) design, where the permutation layer is a bit permutation. Due to the similarity of their design, we give a generic description of such cipher design. Later we will mention the specific components.

We denote such a block cipher by E which operates on n -bit state and κ -bit key. We often represent the internal state of E as a array of nibbles (4-bit word), i.e. $n = 4m$ for some non-negative integer m .

The round function of E comprises three main steps: the substitution (SBox) layer, the permutation layer, and the round key addition.

- The substitution layer is denoted by $\mathcal{S}$ and it applies a fixed 4-bit substitution box S to each nibbles of the internal state in parallel. The lookup tables for the 4-bit SBox (S) for various ciphers are given in table 2.
- The permutation layer is represented by $\mathcal{P}$ which permutes the n bits of the internal state using a permutation function.
- In round key addition, n -bit round key is XORed with the n -bit internal state, where the round key is obtained by some cipher-specific key schedule.

With these definitions, the ℓ -th round function of E where the index starts from 1, can be expressed as:

$$\mathcal{R}_\ell(\mathbf{x}) = (\mathcal{P} \circ \mathcal{S}(\mathbf{x}) \oplus K^\ell), \text{ where } K^\ell \text{ is the } \ell\text{-th round key.}$$

We introduce some more notations here corresponding to internal state of E for convenience. The i -th nibble of the state before the non-linear layer of $\mathcal{R}_\ell$ is denoted by N_i^ℓ and the j -th bit of N_i^ℓ is denoted by $N_{i,j}^\ell$. Similarly, the i -th nibble of the ℓ -th round key is denoted by K_i^ℓ and the j -th bit of K_i^ℓ is denoted by $K_{i,j}^\ell$. We have considered the right most nibble as the 0-th nibble in this work.

Having discussed the generic description of such a cipher, we now move on to the specific components of these ciphers.

DEFAULT: The cipher DEFAULT was proposed by Baksi *et al.* in ASIACRYPT 2021 [3], which follows the paradigm discussed above. In the case of DEFAULT, the state size and key size are 128 bits, i.e., $n = \kappa = 128$ and it has total of 80 rounds. In the design of DEFAULT, there are two different type of round functions,

Primitive	SBox ($\mathcal{S}$)
DEFAULT-LAYER	0 3 7 E D 4 A 9 C F 1 8 B 2 6 5
DEFAULT-CORE	1 9 6 f 7 c 8 2 a e d 0 4 3 b 5
BAKSHEESH	3 0 6 D B 5 8 E C F 9 2 4 A 7 1

Table 2: SBoxes of the ciphers

which differs only in the choice of the SBox. These two type of round functions are named as DEFAULT-LAYER and DEFAULT-CORE. DEFAULT-LAYER consists of 28 rounds and DEFAULT-CORE has 24 rounds, becomes a total of 80 rounds for the full cipher. The DEFAULT-LAYER is applied both before and after of the DEFAULT-CORE layer. In the case of DEFAULT-LAYER, a special type of SBox is used, namely LS structured SBox, whereas in the DEFAULT-CORE block, a non-LS structured SBox is applied on the state (given in table 2). In the permutation layer, all the 128 bits are permuted through a bit permutation function, exactly same as GIFT-128.

In the initial version of DEFAULT, the same master key is used as the round key in DEFAULT-LAYER and a permutation function is applied on the master key to generate round keys of DEFAULT-CORE. In the updated version, 4 round keys, namely k_0, k_1, k_2, k_3 are generated from the given master key (K) by $k_{i+1} = \mathcal{R}'(\mathcal{R}'(\mathcal{R}'(\mathcal{R}'(k_i))))$ for $i \in \{0, \dots, 3\}$. Here $\mathcal{R}'$ is the modified round function without the key addition layer and a constant addition in Add Round Constant (ARC) layer. For $\mathcal{R}'$, the value of the constant is, ‘1’ in the 127 bit and ‘0’ in the other bits of the state. Finally, $K^i = k_{i \bmod 4}$ is used as the i -th round key in DEFAULT cipher. Also, the state is XORed with a 6 bit round constant along with the rounds key in each round.

BAKSHEESH: The cipher BAKSHEESH [4] is also a SPN based structure, which follows the same design paradigm as DEFAULT. Here also block size and key size are 128 bits, i.e., $\kappa = n = 128$ and it has total of 35 rounds. Furthermore, it uses the same bit permutation as DEFAULT and GIFT-128. In the $\mathcal{S}$ layer, it also uses a 4 bit LS SBox, given in table 2.

Before applying the first round, there is an initial key addition, commonly referred to as “pre-whitening”. Here the master key K is used as the pre-whitening key and is denoted as K^0 . Then the round key K^i is generated by a 1 bit circular right shift of the previous round key K^{i-1} , i.e., $K^i = (K^{i-1} \ggg 1)$ for $i \in \{1, \dots, 35\}$. Furthermore, a 6 bit round constant is XORed with the state in each round.

Remark 1. Both DEFAULT and BAKSHEESH employ the permutation layer used in GIFT-128. When considering two consecutive rounds of this permutation layer, it can be decomposed into eight 16-bit permutations, each mapping four nibbles to four nibbles. This structure is known as the “Bad Output must go to Good Input” (BOGI) permutation [5], and the associated groups are referred to as

Quotient-Remainder groups. More specifically, in the ℓ -th round, the j -th Quotient group consists of the nibbles $(N_{4j}^\ell, N_{4j+1}^\ell, N_{4j+2}^\ell, N_{4j+3}^\ell)$, corresponding to all nibble indices whose quotient upon division by 4 is j , for $j \in \{0, \dots, 7\}$. In the following round, the j -th Remainder group includes the nibbles $(N_j^{\ell+1}, N_{j+8}^{\ell+1}, N_{j+16}^{\ell+1}, N_{j+24}^{\ell+1})$, comprising all nibble indices with remainder j when divided by 4, for $j \in \{0, \dots, 7\}$.

2.3 Division Property Based Integral Distinguisher

In FSE 2016, Todo and Morii introduced the bit-based division property [29], which provides a systematic way to determine whether a particular monomial is contained in any monomial of the polynomial representation corresponding to the output bits of a cryptographic function. Below we present the original definition of bit-based division property.

Definition 1. (*Bit-based Division Property.* [29]) *A multiset $X \subseteq \mathbb{F}_2^n$ is said to have the division property $\mathbb{K}$ for some set of n -dimensional vectors $\mathbb{K}$ if for all $\mathbf{u} \in \mathbb{F}_2^n$, it fulfills the following conditions:*

$$\bigoplus_{\mathbf{x} \in X} \mathbf{x}^{\mathbf{u}} = \begin{cases} \text{unknown, if there is } \mathbf{k} \in \mathbb{K} \text{ s.t. } \mathbf{u} \succeq \mathbf{k} \\ 0, \text{ otherwise} \end{cases}$$

The bit-based division property is an effective tool for determining whether a particular monomial is present (the *unknown* case) or absent (the *zero* case) in the polynomial representation of a function. Consider a function $F : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$ and let X be the input set while $Y = \{\mathbf{y} = F(\mathbf{x}) | \mathbf{x} \in X\}$ denotes the corresponding output set. If the input set X has the division property $\mathbf{k}_{in}$ which propagates to the output property $\mathbf{k}_{out}$ of Y , then, according to the definition of bit-based division property, we have

$$\bigoplus_{\mathbf{y} \in Y} \mathbf{y}^{\mathbf{u}} = 0, \forall \mathbf{u} \not\succeq \mathbf{k}_{out}.$$

In particular, if we have a output division property $\mathbf{k}_{out}$, such that the i -th unit vector $\mathbf{e}_i \not\succeq \mathbf{k}_{out}$, then the i -th bit of F is *balanced*, i.e., the occurrence of the *zero* case in the division property ensures that a particular bit is *balanced* in this case. This balanced property can therefore be used as a distinguisher for the cryptographic function.

Assume that, for a given function $F : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$, the i -th output bit is balanced. To construct a distinguisher for F , we first generate a set X of plaintexts that forms an affine subspace determined by the input division property $\mathbf{k}_{in}$. Specifically, for each $\mathbf{x} \in X$, we assign the j -th coordinate to a fixed constant $c_j \in \{0, 1\}$ if the j -th coordinate of $\mathbf{k}_{in}$ is 0, otherwise if the j -th coordinate of $\mathbf{k}_{in}$ is 1, x_j can take any value in $\{0, 1\}$ (if the j -th coordinate of $\mathbf{k}_{in}$ is 1, then we call the j -th bit is *active*). In this way, X constitutes an affine vector space

of dimension $wt(\mathbf{k}_{in})$, where $wt(\mathbf{k}_{in})$ denotes the Hamming weight of $\mathbf{k}_{in}$. This gives rise to the following distinguisher:

$$\bigoplus_{\mathbf{y} \in Y} y_j = \bigoplus_{\mathbf{x} \in X} F(\mathbf{x})_j = 0.$$

The above equations always holds for the function F as a direct consequence of the bit-based division property, whereas it is satisfied with probability $1/2$ for a random function. This distinguisher is referred to as a *zero-sum distinguisher* [2,10,11]. It is important to note that, since the size of X is $2^{wt(\mathbf{k}_{in})}$, the data complexity associated with this division property-based distinguisher is $2^{wt(\mathbf{k}_{in})}$.

Propagation Of Bit-based Division Property: The first step of a division property-based attack is to determine the division property of a reduced-round cipher. To achieve this, it is necessary to analyze how the division property propagates through the basic operations of the cipher. If the input set with division property $\mathbf{k}_0$ propagates to the output set with division property $\mathbf{k}_1$ via a given function, we refer to $(\mathbf{k}_0, \mathbf{k}_1)$ as a valid *division trail*. We recall the formal definition of a valid division trail from [32].

Definition 2. Let f_r denote the round function of an r round iterative primitive. Suppose the initial division property is $\{\mathbf{k}_0\}$ and after $(i-1)$ -round propagation, the division property is $\mathbb{K}_i$. Then we have the following chain of division property propagations: $\{\mathbf{k}_0\} := \mathbb{K}_0 \xrightarrow{f_0} \mathbb{K}_1 \xrightarrow{f_1} \mathbb{K}_2 \xrightarrow{f_2} \dots$. Moreover, for any vector $\mathbf{k}_i \in \mathbb{K}_i$ ($i \geq 1$), there must exist a vector $\mathbf{k}_{i-1} \in \mathbb{K}_{i-1}$ such that $\mathbf{k}_{i-1}$ can propagate to $\mathbf{k}_i$ by division property propagation rules. For $(\mathbf{k}_0, \mathbf{k}_1, \dots, \mathbf{k}_{r-1})$, if $\mathbf{k}_{i-1}$ can propagate to $\mathbf{k}_i$ for all $i \in \{1, 2, \dots, r\}$, we call $(\mathbf{k}_0, \mathbf{k}_1, \dots, \mathbf{k}_r)$ an r -round division trail.

Although finding the division trail is the most critical part of the attack, enumerating all possible division trails for modern ciphers is often a tedious task. A direct programming approach was used in [30] to find the bit-based division properties of SIMON-32 and SIMECK-32, both having a block size of 32 bits. However, this direct approach does not scale to the larger block sizes. In such cases, automatic tools become essential. The main idea is to transform the search for bit-based division properties into a mathematical problem that can be solved by an automatic tool.

Along these lines, Xiang *et al.* [33] first proposed the use of a Mixed Integer Linear Programming (MILP) based tool. Sun *et al.* [28] suggested an alternative approach using SAT/SMT modeling. In fact, DiFA paper [23] also employs the MILP approach to identify valid division trails for the GIFT and PRESENT ciphers. In this paper, we adopt the same MILP-based technique as described in DiFA paper to find valid division trails for the DEFAULT and BAKSHEESH ciphers. For more information on the methodology, please refer to [33] and the original DiFA paper [23]. However, in this work our primary focus is on extending the bit-based division property obtained from MILP, leveraging the algebraic properties of LS SBoxes.

2.4 Longer Division Trail Exploiting LS SBox-es

In this section, we investigate the extension of the bit-based division property by considering whether a division trail identified for a certain number of rounds can be prolonged by exploiting the algebraic structure of the SBox-es. In [24], Lambin *et al.* proposed a method for such an extension based on composing linear layers at the input and output. The primary motivation behind these extensions arises from the fact that division properties are not invariant under linear transformations as discussed in [12]. In other words, for a given cipher E , the approach in [12] recommends analyzing $L_{out} \circ E \circ L_{in}$ instead of E where both L_{in} and L_{out} are strategically chosen linear mappings applied to the input and output, respectively. In this way, it might be possible to find some distinguisher on $L_{out} \circ E \circ L_{in}$ which is not attainable on E . However, the search space for these linear combinations is prohibitively large, necessitating efficient strategies for its reduction. It was suggested in [12] that identifying suitable linear combinations of the output bits, rather than searching over all linear maps, is sufficient for discovering integral distinguishers. In fact, multiplying the output vector by a matrix is equivalent to considering all possible linear combinations of its components. In [18], the authors provided a formal way to find the optimal number of linear combinations that need to be considered to prove non-existence of such extensions, using Depth First Search (DFS).

In this work, we focus on applying linear combination of the output bits to the SBox-es of a given cipher. Suppose we are given a block cipher E where the round function is similar to that of DEFAULT or BAKSHEESH, that is, each round consists of a non-linear operation formed by the parallel application of s -bit SBoxes, followed by a linear layer. Now, assume that we have identified a valid division trail for r rounds, such that some or all output bits are balanced. However, when analyzing the next round, we may observe that the output bits are no longer balanced. This is an outcome that commonly arises due to the non-linear layer.

However, if we can find pairs $\alpha, \beta \in \mathbb{F}_2^n$ (often called as ‘linear mask’) such that $\alpha \cdot \mathbf{x} + \beta \cdot S(\mathbf{x}) = 0$, it becomes possible to extend the division trail to $r + 1$ rounds. It can be readily shown that

$$\bigoplus_{\{\mathbf{y}=S(\mathbf{x}):\mathbf{x}\in X\}} (\beta \cdot \mathbf{y}) = \bigoplus_{\{\mathbf{x}\in X\}} (\alpha \cdot \mathbf{x}).$$

Thus, if we can ensure that the input variables $\{x_i : \alpha_i \neq 0\}$ are balanced over some input set X , then the linear combination $\beta \cdot \mathbf{y}$ of the output bits is also balanced.

We illustrate the basic idea using the example of the LS SBox in DEFAULT. Let the DEFAULT-LAYER cipher’s SBox be defined as $S : \mathbb{F}_2^4 \rightarrow \mathbb{F}_2^4$, where $S(x_3, x_2, x_1, x_0) = (y_3, y_2, y_1, y_0)$. The algebraic normal forms (ANFs) of its four

coordinates are given in eq. (1):

$$\begin{aligned}
y_0 &= x_0 + x_1 + x_2 \\
y_1 &= x_0 + x_1 + x_1x_0 + x_2x_0 + x_3x_1 + x_3x_2 \\
y_2 &= x_1 + x_2 + x_3 \\
y_3 &= x_1x_0 + x_2 + x_2x_0 + x_3 + x_3x_1 + x_3x_2
\end{aligned} \tag{1}$$

It is easy to verify that the following relation holds from the above equations:

$$y_1 \oplus y_2 \oplus y_3 = x_0 \tag{2}$$

Hence, if after r rounds of DEFAULT-LAYER the 0-th input bit of some SBox in the $(r + 1)$ -th round is balanced, then the sum of the first three output bits of that SBox is also balanced. In this way, the division trail can be extended to $(r + 1)$ rounds, provided that the 0-th input bit balanced for some SBox in the $(r + 1)$ -th round.

Obtaining a linear relation such as that in eq. (2) between input and output bits is not surprising when we examine the Linear Approximation Table (LAT) of the SBox of interest. From the LAT of the DEFAULT SBox, given in table 4, we see that there are a total of 7 pairs (α, β) satisfying $\alpha \cdot \mathbf{x} + \beta \cdot \mathbf{y} = 0$ with probability one. The specific equations are given in eq. (3). The above example given in eq. (2) is corresponding to $(\alpha, \beta) = (0\mathbf{x}1, 0\mathbf{x}\mathbf{e})$ (also can be observed in eq. 3). Similarly, from the LAT of the BAKSHEESH SBox in table 3, we observe that there is exactly one LS, corresponding to $(\alpha, \beta) = (0\mathbf{x}8, 0\mathbf{x}7)$. The equation is given in eq. (4).

Δ	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
δ	1	4	-4						-4	-4					
2		4		-4				4					4		
3				-4	-4					4				-4	
4								4	-4	4			4		
5		-4	-4					-4						4	
6			4	4				-4	4						
7				4	-4			-4	-4						
8							8								
9		-4	4							-4	-4				
a	-4		4					4			4				
b	-4	-4						-4		4					
c								4		4	-4	4			
d		-4	-4					4					-4		
e	4		4							4				-4	
f	-4	4										-4	-4		

Table 3: LAT of BAKSHEESH SBox

Δ	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
δ	1													8	
2		4		-4			4			4					
3		4				4	-4						4		
4			4			4		-4				4			
5			4				-4	4					4		
6															8
7															
8											8				
9						8									
a		4					4	4					-4		
b			-4				4		4		4				
c			-4				4	4		4				4	
d			4				4			4			-4		
e				8											
f												8			

Table 4: LAT of DEFAULT-LAYER SBox

2.5 Fault Attacks

A fault attack is a cryptanalytic technique where an attacker deliberately introduces errors (also known as “faults”) in an intermediate round to extract some secret information, like key or intermediate state bits of a cipher. This basically exploits the weakness in the hardware implementation of the cryptosystem. A fault attack mainly depends upon two components, fault injection and fault propagation. In the fault injection process, the attacker injects a fault on the hardware device depending upon the fault model. The fault can be categorized into bit, nibble, or byte level, depending upon the location of the fault and its distribution can be uniform or random. The fault then propagates through the cipher based on its structural properties. The attacker then recovers the secret information after collecting the ciphertexts in the final round.

Differential Fault Attack (DFA). A Differential Fault Attack combines traditional differential cryptanalysis with fault attack technique. Here the attacker first chooses a plaintext (m) and obtains its ciphertext (c). The same plaintext is then passed through the encryption function again. This time, the attacker introduces a fault (δ) at some intermediate r -th round and computes the faulty ciphertext (c'). Consider the cipher has total R number of rounds. Observe that, after applying the fault attack, the cipher can be seen as a $(R - r)$ reduced round cipher where the difference δ is given in the initial round. The attacker then applies the classical differential cryptanalysis technique to get the secret information from ciphertext, faulty-ciphertext pair (c, c') .

Integral Fault Attack (IFA). The Integral Fault Attack (IFA) is the combination of the classical integral cryptanalysis with fault attack. Like the classical one, here also, the attacker wants to generate the All property by giving all possible values in a specific location. The specific position can be bit, byte or nibble, depending upon the structure of the cipher. If the position contains n bits, then after generating the original ciphertext C , the attacker has to give $2^n - 1$ number of faults to generate to the All property in some intermediate round. Suppose this generates the faulty ciphertexts C_i 's at the output. Now, the idea is, from the values of C and C_i 's, the attacker wants to guess the secret information of the cipher. In section 2.3, we have looked at the division property, which is basically the generalization of the integral attack. In the following section, we will look into the BIZ property, which is the propagation of the division property through the cipher.

3 BIZ on LS SBox

As discussed in section 2.3, once a division trail is established for a specified number of rounds, the attacker constructs an affine subspace $X \subset \mathbb{F}_2^n$ corresponding to the input division property. This set is selected such that the corresponding output set exhibits balanced bits, which can then be exploited for key-recovery. Specifically, for each $\mathbf{x} \in X$, the j -th coordinate is fixed to a constant $c_j \in \{0, 1\}$

if the j -th bit of $\mathbf{k}_{\text{in}}$ is 0, otherwise, x_j may take all possible values in $\{0, 1\}$. In the context of integral fault attacks, such input sets are constructed in the intermediate states of the cipher using faults, as discussed in section 2.5. This step is critical for the success of the attack, as each repetition (which is often necessary to increase the filtering effect) requires faults to be injected at exactly the same locations in the intermediate state during every iteration. For example, in all our attacks, we consider input division properties with four active bits corresponding to a single SBox, i.e., one active nibble.

In this direction, Kundu et al. [23] introduced a novel property of GIFT-like structures. They observed that, regardless of which nibble is active in the input division property, certain fixed balanced bits consistently appear in the output division property. Specifically, for the PRESENT cipher, all bits become balanced after 4 rounds, while 4 particular bits remain balanced after 5 rounds. For GIFT-64, 64 and 16 bits become balanced after 4 and 5 rounds, respectively. In the case of GIFT-128, two different sets of 80 bits appear after 5 rounds, of which 64 bits are always balanced, provided the initial input division property is assigned to N_i for $i \in \{0, \dots, 31\}$. Utilizing this property, the authors performed the fault attack at $\mathcal{R}_{25}$ for PRESENT and $\mathcal{R}_{19}$ for GIFT-64.

Subsequently, Ghosh et al. [19] investigated the underlying reason for the emergence of this property. By employing the ANF representation, they demonstrated that the phenomenon results from the combined effect of both the SBox and permutation layers. This characteristic was termed the *Bit Invariance Zero-sum* (BIZ) property. Moreover, they confirmed that BIZ manifests after 5 rounds of DEFAULT, where the number of balanced bits is 128. In the following, we extend the BIZ property to LS structured SBox with the idea proposed in section 2.4.

3.1 7 rounds Distinguisher of DEFAULT

In this work, we first re-verified the BIZ property for 5 rounds of the DEFAULT cipher and further observed that this property also holds after 6 rounds. Specifically, after 6 rounds, there are 64 bits, namely, $N_{i,0}$ and $N_{i,2}$ for each nibble $i \in \{0, \dots, 31\}$, for which the BIZ property persists when the input division property is provided at any nibble N_j with $j \in \{0, \dots, 31\}$. Notably, this 6-round manifestation of the BIZ property has not been previously reported in [19].

Additionally, based on the study of the LAT of the SBox of DEFAULT, as discussed in section 2.4, it can be seen that the 6-round BIZ distinguisher can be extended by one round using the following set of 7 linear equations:

$$\begin{aligned} x_0 \oplus y_1 \oplus y_2 \oplus y_3 &= 0, x_1 \oplus x_2 \oplus y_0 \oplus y_1 \oplus y_2 \oplus y_3 = 0, x_0 \oplus x_1 \oplus x_2 \oplus y_0 = 0 \\ x_3 \oplus y_3 \oplus y_1 \oplus y_0 &= 0, x_3 \oplus x_0 \oplus y_2 \oplus y_0 = 0, x_3 \oplus x_2 \oplus x_1 \oplus y_2 = 0 \\ x_3 \oplus x_2 \oplus x_1 \oplus x_0 \oplus y_3 \oplus y_1 &= 0 \end{aligned} \tag{3}$$

Although there are seven such equations for DEFAULT, only the first equation can be used to extend the 6-round BIZ distinguisher. This is because all other

equations involve either x_1 or x_3 , which are not balanced after 6 rounds. Consequently, by combining the 6-round BIZ distinguisher with this one-round linear equation for the SBox, we obtain a distinguisher covering a total of 7 rounds as shown in fig. 1. We employ this extended distinguisher in our subsequent key recovery attack on DEFAULT.

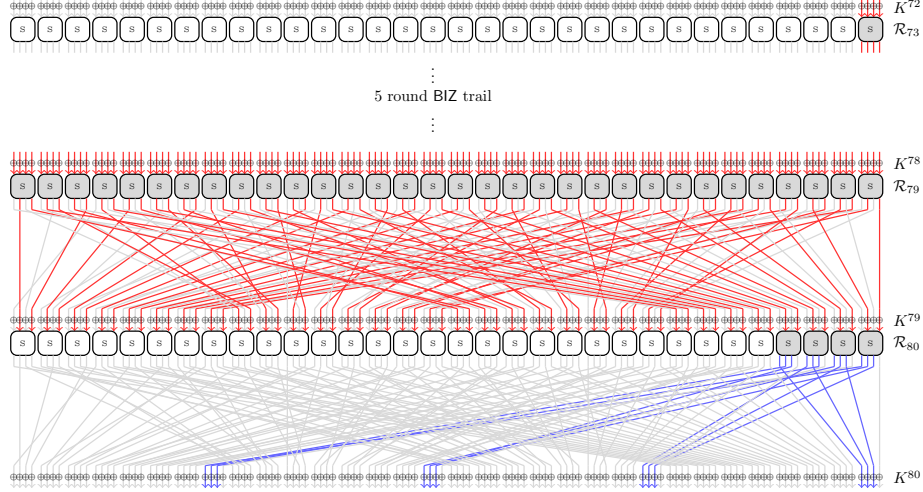


Fig. 1: 7 rounds linear distinguisher of DEFAULT

3.2 6 Round Distinguisher of BAKSHEESH

It was also reported in [19] that the BIZ property holds for up to 5 rounds in the case of BAKSHEESH, with all 128 bits becoming balanced after 5 rounds. Moreover, an examination of the LAT for the SBox used in BAKSHEESH reveals the following linear relation between input and output bits:

$$x_3 \oplus y_0 \oplus y_1 \oplus y_2 = 0 \quad (4)$$

Applying the same approach as in the case of DEFAULT, we can thus extend the zerosum distinguisher in BAKSHEESH by one additional round (6-round distinguisher in total) by leveraging this linear relation for the SBox. The resulting distinguisher is employed in our subsequent key recovery attack on BAKSHEESH.

4 Key-Recovery Attack Using BIZ

In this section, we present key-recovery attacks on the DEFAULT and BAKSHEESH ciphers based on the Integral Fault Attack (IFA). We begin by describing a general framework applicable to our key-recovery strategy. Let E denote an

r -round block cipher with an n -bit state and a k -bit key, structured as detailed in section 2.2. Let the cipher be partitioned such that the first r_f rounds are denoted by E_f , and the subsequent r_d rounds by E_d . Suppose a distinguisher exists for r_d rounds, i.e., for E_d , such that there is an affine subspace $X \subseteq \mathbb{F}_2^n$ where a linear combination of the output bits is balanced over X . Formally, for a given $\beta \in \mathbb{F}_2^n$, we have:

$$\bigoplus_{\mathbf{x} \in X} (\beta \cdot E_d(\mathbf{x})) = \mathbf{0}.$$

Once such a distinguisher is identified, it can be utilized for key-recovery by extending the cipher with r_k additional rounds denoted by E_k , i.e., $r_f + r_d + r_k = r$. For notational clarity, we ignore the final linear layer (if present), since its omission does not impact the attack's effectiveness. Thus, the final round can be described as $\mathcal{R}_r(\mathbf{x}) = \mathcal{S}(\mathbf{x}) \oplus K^r$ and the cipher can be expressed as $E = E_k \circ E_d \circ E_f$.

Let $Y = \{E_d(\mathbf{x}) \mid \mathbf{x} \in X\}$ denote the set of ciphertexts corresponding to the encryption of the affine subspace X . The core idea of the key-recovery phase is to guess portions of the relevant round keys and to partially decrypt the corresponding segments of ciphertexts in Y through the last round function $\mathcal{R}_r$. Subsequently, one verifies whether the XOR sum of certain linear combinations of partially decrypted values is equal to zero.

Typically, it is computationally advantageous to perform partial decryption on selected segments of the ciphertext, rather than the full ciphertext, thereby reducing the search space for the key. For example, in all our attacks, we conduct partial decryption on one **Remainder** group at a time. Assume we seek to partially decrypt χ bits of each ciphertext, which corresponds to guessing χ bits of the subkey. Let $[\mathbf{y}]_\chi$ denote a specific subset of coordinates of $\mathbf{y} \in \mathbb{F}_2^n$ to be partially decrypted. For each χ -bit subkey guess K , we check the following condition:

$$\bigoplus_{\mathbf{x} \in X} (\beta \cdot E^{-1}(K, [E(\mathbf{x})]_\chi)) = \bigoplus_{\mathbf{y} \in Y} (\beta \cdot [\mathbf{y}]_\chi) = \bigoplus_{\mathbf{x} \in X} (\beta \cdot E_d(\mathbf{x})) = \mathbf{0}. \quad (5)$$

If the key guess K is not correct, the resulting sum will not satisfy the balanced property, thus filtering out the incorrect subkey from the set of candidates.

4.1 Key-Recovery Attack on BAKSHEESH

Building on the general framework introduced previously, we now detail the key-recovery attack on the BAKSHEESH cipher, leveraging the 6-round distinguisher presented in section 3.2 (i.e., $r_d = 6$). To construct the affine subspace X of size 2^4 , we inject a random nibble fault at N_j^{28} in $\mathcal{R}_{28}$, for a fixed but randomly chosen $j \in \{0, \dots, 31\}$. One element of X corresponds to the fault-free state, while the remaining elements are produced via fault injection. In accordance with the Coupon Collector's principle, the replay count increases as we repeat the attack to exhaust all values of the input set. After applying the *ARC* and inverse permutation layers, the collection of faulty and fault-free ciphertexts is stored in $\mathcal{C}_j$.

The primary objective is to recover the last round key K^{35} . However, since the permutation layer is inverted prior to analysis, we equivalently aim to recover $U^{35} = \mathcal{P}^{-1}(K^{35})$. The key-recovery procedure exploits the Quotient-Remainder group structure (please refer to remark 1) of BAKSHEESH's permutation layer $\mathcal{P}$ by performing partial decryption of $\chi = 16$ bits of each ciphertext in $\mathcal{C}_j$ corresponding to one Remainder group at a time. Rather than targeting the full keyspace, we iteratively reduce the keyspace of each Remainder remainder group. There are eight Remainder groups in total, each comprising 16 bits. For every possible 2^{16} subkey within a Remainder group, a counter, *alive*, is maintained, indicating whether a candidate subkey is still viable.

For illustration, consider the 0-th Remainder group, we guess 16 bits of U^{35} , denoted by u_0 , associated with nibbles $(U_0^{35}, U_8^{35}, U_{16}^{35}, U_{24}^{35})$. For every candidate u_0 , we partially decrypt $\chi = 16$ bits of the ciphertexts in $\mathcal{C}_j$ and check whether the balanced property, as expressed in eq. (5), holds. If the XOR sum is zero, u_0 is recorded as a possible key for the 0-th Remainder group. This process is repeated for each of the Remainder groups, updating the corresponding *alive*[i] counters corresponding to the i -th Remainder group. The full procedure for the 0-th remainder group is illustrated in fig. 2. This part of the attack is presented in algorithm 1.

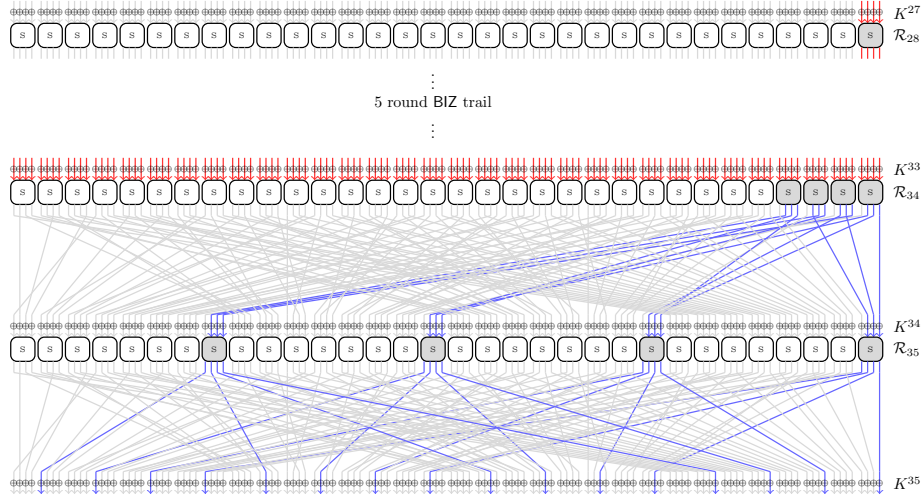


Fig. 2: A 7-rounds key-recovery using 6-round extended distinguisher of BAKSHEESH

We observe that, for a single input set X , the keyspace for each remainder group is reduced from 2^{16} to 2^{12} , effectively recovering 4 bits of key information per Remainder group. Repeating the attack with additional input sets X further reduces the keyspace, after 6 repetitions, the keyspace for each group reaches 2^4 .

Notably, increasing the number of repetitions beyond 6 fails to further reduce the keyspace below 2^4 . At this stage, the total reduced keyspace is 2^{32} . To refine the keyspace further, we employ the relation between two round keys, as described in the following section.

Algorithm 1 CHECKDISTINGUISHER

Input: m arrays of ciphertexts corresponding to m input sets, $\mathcal{C}_i$ for $i \in \{0, \dots, 5\}$ with $|\mathcal{C}_i| = 16$
Output: 8 arrays 16-bit possible keys, one for each Remainder group

- 1: Initialize empty arrays $alive_r, r \in \{0, \dots, 7\}$, one for each Remainder group
- 2: **for** $i = 0$ to $m - 1$ **do**
- 3: **for** $r = 0$ to 7 **do** ▷ for each Remainder group
- 4: **for** $k \in \{0, \dots, 2^{16} - 1\}$ **do**
- 5: $sum \leftarrow 0$
- 6: **for all** $c \in \mathcal{C}_i$ **do**
- 7: $a \leftarrow c \oplus k$
- 8: $a \leftarrow \mathcal{S}^{-1}(a)$
- 9: $sum \leftarrow sum \oplus a$
- 10: $Q \leftarrow$ The index corresponding to r -th Quotient group
- 11: $flag \leftarrow 1$
- 12: **for all** $q \in Q$ **do**
- 13: **if** $(sum_{q,1} \oplus sum_{q,2} \oplus sum_{q,3}) \neq 0$ **then**
- 14: $flag \leftarrow 0$
- 15: **break**
- 16: **if** $flag = 1$ **then**
- 17: $alive_r \leftarrow alive_r \cup \{k\}$
- 18: **return** $alive_r$, for $r \in \{0, \dots, 7\}$

Master-Key Recovery: While the above approach significantly reduces the keyspace, it ultimately leaves 2^4 possibilities in each Remainder group, even after multiple repetitions. To uniquely determine the master key, a further refinement of the attack is required. Our strategy is to partially decrypt through an additional SBox layer, which introduces further nonlinearity and thus increases the filtering effect. To enable this additional partial decryption layer, knowledge of K^{34} is required. Rather than guessing the key nibbles of K^{34} directly, we reconstruct them from U^{35} using the key schedule of BAKSHEESH, which gives the following equation $\mathcal{P}(U^{35}) = K^{35} = (K^{34} \ggg 1)$. This key-relation allows us to infer portions of K^{34} from candidate subkeys for U^{35} .

A naive approach is to combine the candidate keys from all the Remainder groups in order to reconstruct the full 128-bit U^{35} . With this complete key, one may then compute K^{34} as $K^{34} = \mathcal{P}(U^{35}) \lll 1$ using the key schedule relation. For each such pair (U^{35}, K^{34}) , it is possible to invert all ciphertexts through the

SBox layer and test for the distinguishers at the output of $\mathcal{R}_{33}$. However, the computational efficiency can be improved by focusing on only a subset of the Remainder groups, as detailed below.

To maximize the effectiveness of this step, we identify combinations of Remainder groups that allow us to invert the maximum possible number of SBox in $\mathcal{R}_{34}$. This process is illustrated in fig. 3, where we focus on the Remainder groups 0, 3, 4, and 7, collectively forming a 64-bit segment of U^{35} . Applying the corresponding bit permutation and a one-bit left rotation, this segment maps to a 64-bit portion of K^{34} . We are especially interested in those bits of K^{34} that correspond to all four output bits of a single SBox, as this enables us to invert that SBox entirely. For instance, for some nibbles, we obtain only three or one (see N_1^{34} and N_3^{34} , colored in green in fig. 3) corresponding key bits, whereas for 15 specific nibbles at $\mathcal{R}_{34}$, namely, N_j^{34} for $j \in \{0, 4, 5, 8, 9, 12, 13, 16, 17, 20, 21, 24, 25, 28, 29\}$, all four key bits are known. It is also crucial, during partial decryption through $\mathcal{R}_{34}$, to consider for which

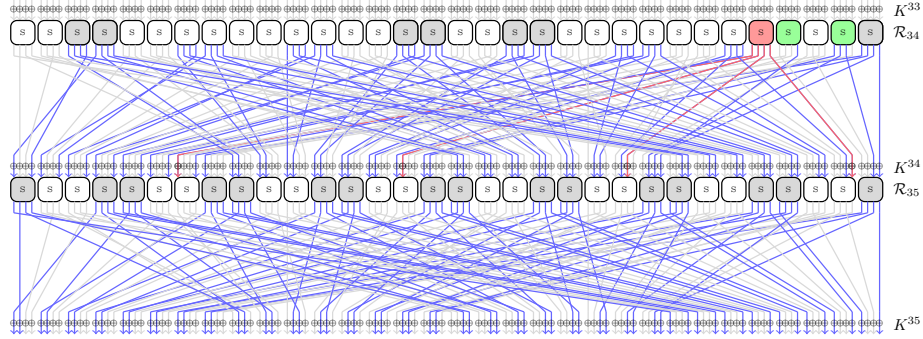


Fig. 3: key relation for 2 rounds of BAKSHEESH

SBoxes the key bits of U^{35} are already known. For example, although the key bits corresponding to N_4^{34} are known, decryption also requires the bit $K_{1,0}^{35}$, which has not been guessed (illustrated in red in the fig. 3); thus, the SBox at N_4^{34} cannot be inverted to check the distinguisher at the output of $\mathcal{R}_{33}$. As a result, only certain S-boxes can be inverted after guessing the Remainder groups 0, 3, 4, and 7 at $\mathcal{R}_{35}$, specifically, N_j^{34} for $j \in \{0, 12, 13, 16, 17, 28, 29\}$, as shown in grey in fig. 3.

We have systematically explored various combinations involving 2, 3, or 5 Remainder groups, and have found that selecting 4 groups offers the best trade-off between known key bits and computational complexity. In particular, the combinations $\{0, 3, 4, 7\}$ and $\{1, 2, 5, 6\}$ allow us to invert 7 and 8 SBoxes, respectively, in $\mathcal{R}^{34}$. This technique enables the recovery of the unique master key from the previously reduced key space.

The complete key-recovery procedure is outlined in algorithm 2, which invokes algorithm 1 to identify the remaining candidate keys after the initial filtering step using the extended distinguisher for each **Remainder** group. In algorithm 2, candidate keys from remainder groups 0, 3, 4, and 7 are combined to form 64-bit keys, denoted as $\mathcal{G}_1$, while those from remainder groups 1, 2, 5, and 6 are combined to form another set of 64-bit keys, $\mathcal{G}_2$. For each candidate in $\mathcal{G}_1$ and $\mathcal{G}_2$, the BIZ distinguisher is checked at $\mathcal{R}^{33}$. Specifically, for keys in $\mathcal{G}_1$, the distinguisher is verified at N_j^{33} for $j \in \{0, 12, 13, 16, 17, 28, 29\}$ (see step 16 in algorithm 2), while for keys in $\mathcal{G}_2$, the distinguisher is checked at N_j^{33} for $j \in \{6, 7, 10, 11, 22, 23, 26, 27\}$ (again, see step 16). The candidate key sets $\mathcal{G}_1$ and $\mathcal{G}_2$ are then merged to construct possible combinations for K^{35} and K^{34} , the keys for the final two rounds. At this stage, all remainder groups have been combined, allowing for a check of the full BIZ distinguisher at the input to the S-box layer in $\mathcal{R}^{33}$, ultimately yielding the final candidate key space $\mathcal{K}$. Simulation results indicate that, after six repetitions, the key space for each of $\mathcal{G}_1$ and $\mathcal{G}_2$ is reduced to approximately 2^{11} candidates, and the final key space $\mathcal{K}$ is reduced to just 2^1 on average. This multi-step combination approach effectively keeps the overall time complexity of the attack manageable.

Complexity of Our Attack. Our attack proceeds in two main stages. In the first stage, we independently reduce the key space for each **Remainder** group. In the second stage, we determine the master key by combining the reduced key spaces of the **Remainder** groups.

The time complexity of our attack is assessed in terms of the number of SBox inversions. In the first stage, we examine all 2^{16} candidate keys for each **Remainder** group. Since all groups can be processed in parallel, the total cost for this stage is $2^{16} \times 4 \times 96 \approx 2^{25}$ SBox inversions. Upon completion, the key space for each **Remainder** group is reduced to 2^4 . In the second stage, we combine four **Remainder** groups at a time to generate the candidate sets $\mathcal{G}_1$ and $\mathcal{G}_2$, each comprising 2^{16} possible keys and containing information on 8 nibbles. These sets can again be processed in parallel, so the time complexity for each is $2^{16} \times 8 \times 96 \approx 2^{26}$ SBox inversions, reducing $\mathcal{G}_1$ and $\mathcal{G}_2$ to approximately 2^{11} candidates each. For the final candidate key space, we combine these results, leading to $2^{11} \times 2^{11} = 2^{22}$ possible keys. At this stage, we must invert the entire state (or equivalently, two rounds of the cipher) for each candidate. Thus, the final time complexity is $2^{22} \times 32 \times 2 \times 96 = 2^{35}$ SBox inversions, which is approximately equivalent to 2^{30} full-round inversions.

As for memory complexity, we need to store the ciphertexts for the 6 input sets, with each ciphertext represented as a 16-bit value for each **Remainder** group. This requires storage for $6 \times 24 \times (8 \times 16) \approx 213.58$ bits. Additionally, we need to store 2^4 key suggestions for each **Remainder** group during stage one and 2×2^{11} key suggestions in stage two, for a total of $8 \times 2^4 + 2 \times 2^{11} \approx 2^{12.04}$ bits.

4.2 Key Recovery Attack on DEFAULT

We now discuss the key-recovery attack on DEFAULT using the 7-round integral distinguisher of DEFAULT presented in section 3.1. This attack is similar to the attack on BAKSHEESH, described in section 4.1. Here each SBox generates one equation between its input and output bits, totaling of 32 equations. Similar to the previous attack on BAKSHEESH, to generate an affine subspace we consider a random nibble j before the SBox layer at $\mathcal{R}_{72}$ i.e. N_j^{72} to inject faults. Here also we omit the last liner layer, i.e., after applying the *ARC* and inverse permutation layers, the collection of faulty and fault-free ciphertexts is stored in $\mathcal{C}_j$. Now, to reduce the keyspace of the last round, here also we use the Quotient-Remainder group structure of the permutation layer ($\mathcal{P}$) of DEFAULT. Due to the similarity with previous attack we skip low level details of this attack. The key-recovery for the 0-th Remainder group on DEFAULT is depicted in fig. 4.

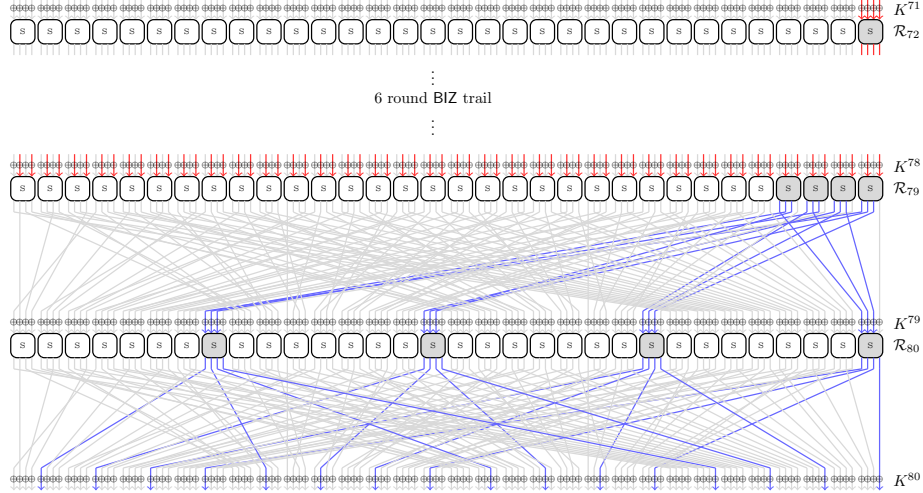


Fig. 4: 8 rounds key recovery attack using 7 round linear distinguisher of DEFAULT

We have observe that, using 1 input set, the key space of each reminder group reduces to 2^{12} from 2^{16} possible keys and by using 6 input sets the keyspace of each reminder group reduce to 2^8 . However, the keyspace does not reduce more than 2^8 if we increase the amount of input sets also. Hence, the size of the total reduced keyspace becomes 2^{64} . For the DEFAULT cipher, we could not manage to use key-relations to further reduce the keyspace, due to the non-linear nature of the key-schedule.

Complexity of Our Attack. As with the first step of the attack on BAKSHEESH, we examine all 2^{16} candidate keys for each Remainder group. Since all groups

can be processed in parallel, the total cost for this stage is $2^{16} \times 4 \times 96 \approx 2^{25}$ *SBox* inversions.

As for memory complexity, we need to store the ciphertexts for the 6 input sets, with each ciphertext represented as a 16-bit value for each **Remainder** group. This requires storage for $6 \times 24 \times (8 \times 16) \approx 213.58$ bits.

5 Attack Simulation

To validate the effectiveness of our proposed attacks, we conducted comprehensive simulations on both ciphers. The verification of the **BIZ** distinguishers was carried out by employing Mixed Integer Linear Programming (MILP) methods, as well as by implementing the integral distinguishers (as described in section 3) using custom C code. Both the key-recovery attack procedures — outlined in algorithm 1 and algorithm 2 — were also independently implemented and evaluated through dedicated C programs. All experiments were performed on a machine equipped with an Intel(R) Core(TM) i3-7020U CPU @ 2.30GHz and 12 GB of RAM. Notably, each simulation completed in under five minutes.

To ensure the robustness and consistency of our attack, we repeated the experiments 100 times. In each iteration, we verified that the actual key was retained within the resulting candidate key space. This outcome serves as practical evidence for the validity of the distinguishers and key-recovery attacks presented in this work. The average size of the reduced key space was computed over all 100 runs and is reported in the corresponding sections. All the code used in our experiments is provided at https://anonymous.4open.science/r/difa_extension_simulation-B7EB/README.md and will also be released through an open-source repository following the publication of this paper.

6 Conclusion

In this work, we have explored the possibility of improving fault penetration in LS Sbox based ciphers **BAKSHEESH** and **DEFAULT**. We extensively studied the effect of linear structures of the SBoxes on allowing division trails to be extended by multiple rounds. Using our observation we devised new division trails on both the ciphers. Using these trails we applied the DiFA attack with an improved fault injection point of two more rounds for **BAKSHEESH** and three more rounds for **DEFAULT**. With 6 sets of nibble based faults we reduced the key-spaces to 1 and 2^{64} for **BAKSHEESH** and **DEFAULT** respectively giving new insights into role of LS in SBoxes in extending fault penetration.

References

1. Aikata, Banashri Karmakar, and Dhiman Saha. PRINCE under differential fault attack: Now in 3d. In *ASHES@CCS*, pages 81–91. ACM, 2020.

2. Jean-Philippe Aumasson and Willi Meier. Zero-sum distinguishers for reduced Keccak-f and for the core functions of Luffa and Hamsi. *rump session of Cryptographic Hardware and Embedded Systems (CHES)*, 2009:67, 2009.
3. Anubhab Baksi, Shivam Bhasin, Jakub Breier, Mustafa Khairallah, Thomas Peyrin, Sumanta Sarkar, and Siang Meng Sim. DEFAULT: cipher level resistance against differential fault attack. In *ASIACRYPT (2)*, volume 13091 of *Lecture Notes in Computer Science*, pages 124–156. Springer, 2021.
4. Anubhab Baksi, Jakub Breier, Anupam Chattopadhyay, Tomas Gerlich, Sylvain Guilley, Naina Gupta, Kai Hu, Takanori Isobe, Arpan Jati, Petr Jedlicka, Hyunjun Kim, Fukang Liu, Zdenek Martinasek, Kosei Sakamoto, Hwajeong Seo, Rentaro Shiba, and Ritu Ranjan Shrivastwa. BAKSHEESH: similar yet different from GIFT. *IACR Cryptol. ePrint Arch.*, page 750, 2023.
5. Subhadeep Banik, Sumit Kumar Pandey, Thomas Peyrin, Yu Sasaki, Siang Meng Sim, and Yosuke Todo. GIFT: A small present - towards reaching the limit of lightweight encryption. In *CHES*, volume 10529 of *Lecture Notes in Computer Science*, pages 321–345. Springer, 2017.
6. Eli Biham and Adi Shamir. Differential fault analysis of secret key cryptosystems. In *CRYPTO*, volume 1294 of *Lecture Notes in Computer Science*, pages 513–525. Springer, 1997.
7. Johannes Blömer and Volker Krummel. Fault based collision attacks on AES. In *FDTC*, volume 4236 of *Lecture Notes in Computer Science*, pages 106–120. Springer, 2006.
8. Dan Boneh, Richard A. DeMillo, and Richard J. Lipton. On the importance of checking cryptographic protocols for faults (extended abstract). In *EUROCRYPT*, volume 1233 of *Lecture Notes in Computer Science*, pages 37–51. Springer, 1997.
9. Dan Boneh, Richard A. DeMillo, and Richard J. Lipton. On the importance of eliminating errors in cryptographic computations. *J. Cryptol.*, 14(2):101–119, 2001.
10. Christina Boura and Anne Canteaut. Zero-Sum Distinguishers for Iterated Permutations and Application to Keccak-f and Hamsi-256. In *Proceedings of Selected Areas in Cryptography (SAC)*, volume 6544 of *Lecture Notes in Computer Science*, pages 1–17. Springer, 2010.
11. Christina Boura, Anne Canteaut, and Christophe De Cannière. Higher-Order Differential Properties of Keccak and Luffa. In *Proceedings of Fast Software Encryption (FSE)*, volume 6733 of *Lecture Notes in Computer Science*, pages 252–269. Springer, 2011.
12. P. Derbez and Pierre-Alain Fouque. Increasing precision of division property. *IACR Cryptol. ePrint Arch.*, 2021:22, 2020.
13. Patrick Derbez, Pierre-Alain Fouque, and Delphine Leresteux. Meet-in-the-middle and impossible differential fault analysis on AES. In *CHES*, volume 6917 of *Lecture Notes in Computer Science*, pages 274–291. Springer, 2011.
14. Patrick Derbez and Léo Perrin. Meet-in-the-middle attacks and structural analysis of round-reduced PRINCE. *J. Cryptol.*, 33(3):1184–1215, 2020.
15. Chandan Dey, Sumit Kumar Pandey, Tapabrata Roy, and Santanu Sarkar. Differential fault attack on DEFAULT. *Adv. Math. Commun.*, 18(4):909–921, 2024.
16. Christoph Dobraunig, Maria Eichlseder, Hannes Groß, Stefan Mangard, Florian Mendel, and Robert Primas. Statistical ineffective fault attacks on masked AES with fault countermeasures. In *ASIACRYPT (2)*, volume 11273 of *Lecture Notes in Computer Science*, pages 315–342. Springer, 2018.
17. Thomas Fuhr, Éliane Jaulmes, Victor Lomné, and Adrian Thillard. Fault attacks on AES with faulty ciphertexts only. In *FDTC*, pages 108–118. IEEE Computer Society, 2013.

18. Shibam Ghosh and Orr Dunkelman. Automatic search for bit-based division property. In *LATINCRYPT*, volume 12912 of *Lecture Notes in Computer Science*, pages 254–274. Springer, 2021.
19. Shibam Ghosh, Anup Kumar Kundu, Mostafizar Rahman, and Dhiman Saha. Bizness: Bit invariant zero-sum property based on division trail. In *INDOCRYPT (2)*, volume 15496 of *Lecture Notes in Computer Science*, pages 134–155. Springer, 2024.
20. Amit Jana, Anup Kumar Kundu, and Goutam Paul. More vulnerabilities of linear structure sbox-based ciphers reveal their inability to resist DFA. In *ASIACRYPT (8)*, volume 15491 of *Lecture Notes in Computer Science*, pages 168–203. Springer, 2024.
21. Anup Kumar Kundu, Aikata, Banashri Karmakar, and Dhiman Saha. Fault analysis of the PRINCE family of lightweight ciphers. *J. Cryptogr. Eng.*, 12(4):475–494, 2022.
22. Anup Kumar Kundu, Shibam Ghosh, Aikata Aikata, and Dhiman Saha. Tofa: Towards fault analysis of GIFT and gift-like ciphers leveraging truncated impossible differentials. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2025(3):614–643, 2025.
23. Anup Kumar Kundu, Shibam Ghosh, Dhiman Saha, and Mostafizar Rahman. Divide and rule: Difa - division property based fault attacks on PRESENT and GIFT. In *ACNS (1)*, volume 13905 of *Lecture Notes in Computer Science*, pages 89–116. Springer, 2023.
24. Baptiste Lambin, Patrick Derbez, and Pierre-Alain Fouque. Linearly equivalent S-boxes and the division property. *Designs, Codes and Cryptography*, 88, 2020.
25. Marcel Nageler, Christoph Dobraunig, and Maria Eichlseder. Information-combining differential fault attacks on DEFAULT. In *EUROCRYPT (3)*, volume 13277 of *Lecture Notes in Computer Science*, pages 168–191. Springer, 2022.
26. Dhiman Saha and Dipanwita Roy Chowdhury. Encounter: On breaking the nonce barrier in differential fault analysis with a case-study on PAEQ. In *CHES*, volume 9813 of *Lecture Notes in Computer Science*, pages 581–601. Springer, 2016.
27. Kazuo Sakiyama, Yu Sasaki, and Yang Li. *Security of Block Ciphers - From Algorithm Design to Hardware Implementation*. Wiley, 2015.
28. Ling Sun, Wei Wang, and Meiqin Wang. Automatic search of bit-based division property for ARX ciphers and word-based division property. In *ASIACRYPT (1)*, volume 10624 of *Lecture Notes in Computer Science*, pages 128–157. Springer, 2017.
29. Yosuke Todo and Masakatu Morii. Bit-based division property and application to simon family. In *FSE*, volume 9783 of *Lecture Notes in Computer Science*, pages 357–377. Springer, 2016.
30. Yosuke Todo and Masakatu Morii. Bit-Based Division Property and Application to Simon Family. In *Proceedings of Fast Software Encryption (FSE)*, volume 9783 of *Lecture Notes in Computer Science*, pages 357–377. Springer, 2016.
31. Navid Vafaei, Sara Zarei, Nasour Bagheri, Maria Eichlseder, Robert Primas, and Hadi Soleimany. Statistical effective fault attacks: The other side of the coin. *IEEE Trans. Inf. Forensics Secur.*, 17:1855–1867, 2022.
32. Zejun Xiang, Wentao Zhang, Zhenzhen Bao, and Dongdai Lin. Applying MILP Method to Searching Integral Distinguishers Based on Division Property for 6 Lightweight Block Ciphers. In *Advances in Cryptology – Proceedings of ASIACRYPT*, volume 10031 of *Lecture Notes in Computer Science*, pages 648–678. Springer, 2016.
33. Zejun Xiang, Wentao Zhang, Zhenzhen Bao, and Dongdai Lin. Applying MILP method to searching integral distinguishers based on division property for 6

- lightweight block ciphers. In *ASIACRYPT (1)*, volume 10031 of *Lecture Notes in Computer Science*, pages 648–678, 2016.
34. Fan Zhang, Xiaoxuan Lou, Xinjie Zhao, Shivam Bhasin, Wei He, Ruyi Ding, Samiya Qureshi, and Kui Ren. Persistent fault analysis on block ciphers. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2018(3):150–172, 2018.

Algorithm 2 KEYRECOVERY-BAKSHEESH

Input: m arrays of ciphertexts corresponding to m input sets, $\mathcal{C}_i$ for $i \in \{0, \dots, 5\}$ with $|\mathcal{C}_i| = 16$

Output: A list of possible keys $\mathcal{K}$

```

1: for  $i = 0$  to  $m - 1$  do
2:   for  $c \in \mathcal{C}_i$  do
3:      $c \leftarrow (\mathcal{P}^{-1} \circ \text{ARC})(c),$   $\triangleright$  Invert through the last linear layer
4: Generate  $alive_r$  from  $\text{CHECKDISTINGUISHER}(\mathcal{C}_i)$  with  $i < m$ , for  $r \in \{0, \dots, 7\}$ 

---


    Combining Four Remainder Groups
    

---


5:  $\mathcal{G}_1 \leftarrow \text{Combine}(alive_r),$  for  $r \in \{0, 3, 4, 7\}$ 
6:  $\mathcal{G}'_1 \leftarrow \{\}$ 
7: for all  $u^{35} \in \mathcal{G}_1$  do
8:    $k^{34} \leftarrow \mathcal{P}(u^{35}) \lll 1$   $\triangleright$  Possible second last round key
9:   for  $i = 0$  to  $m - 1$  do
10:     $\text{sum} \leftarrow 0$ 
11:    for  $c \in \mathcal{C}_i$  do
12:       $a \leftarrow \mathcal{S}^{-1}(c \oplus u^{35})$   $\triangleright$  Invert the last SBox layer
13:       $a \leftarrow (\mathcal{S}^{-1} \circ \mathcal{P}^{-1} \circ \text{ARC})(a \oplus k^{34})$   $\triangleright$  Invert second last layer
14:       $\text{sum} \leftarrow \text{sum} \oplus a$ 
15:       $\text{flag} \leftarrow 1$ 
16:      for  $n \in \{0, 12, 13, 16, 17, 28, 29\}$  do
17:        if  $\text{sum}_n \neq 0$  then
18:           $\text{flag} \leftarrow 0$ 
19:          break
20:      if  $\text{flag} = 1$  then
21:         $\mathcal{G}'_1 \leftarrow \mathcal{G}'_1 \cup \{u^{35}\}$ 
22: Use  $r \in \{1, 2, 5, 6\}$  for steps 5 to 21 and  $n \in \{6, 7, 10, 11, 22, 23, 26, 27\}$  in
    step 16 to generate  $\mathcal{G}'_2$ 

---


    Combining All Remainder Groups
    

---


23:  $\mathcal{K} \leftarrow \{\}$ 
24: for each  $(k_1, k_2) \in \mathcal{G}'_1 \times \mathcal{G}'_2$  do
25:   Construct  $K^{35}, K^{34}$  from  $k_A$  and  $k_B$ 
26:   for  $i = 0$  to  $m - 1$  do
27:     $\text{sum} \leftarrow 0$ 
28:    for  $c \in \mathcal{C}_i$  do
29:       $a \leftarrow \text{Decrypt } c \text{ 2 rounds using } K^{35} \text{ and } K^{34}$ 
30:       $\text{sum} \leftarrow \text{sum} \oplus a$ 
31:      if  $\text{sum} = 0$  then  $\triangleright$  BIZ distinguisher at  $\mathcal{R}_{33}$ 
32:         $\mathcal{K} \leftarrow \mathcal{K} \cup \{k\}$ 
33: return  $\mathcal{K}$ 

```
